Lab 02: Basic NLP Preprocessing Techniques

Course: ITAI 2373 - Natural Language Processing
Module: 02 - Text Preprocessing
Duration: 2-3 hours
Student Name: Oman Malek
Date: 08/29/26

🎯 Learning Objectives

By completing this lab, you will:

1. Understand the critical role of preprocessing in NLP pipelines
2. Master fundamental text preprocessing techniques
3. Compare different libraries and their approaches
4. Analyze the effects of preprocessing on text data
5. Build a complete preprocessing pipeline
6. Load and work with different types of text datasets

📖 Introduction to NLP Preprocessing

Natural Language Processing (NLP) preprocessing refers to the initial steps taken to clean and transform raw text data into a format that's more suitable for analysis by machine learning algorithms.

Why is preprocessing crucial?

1. **Standardization:** Ensures consistent text format across your dataset
2. **Noise Reduction:** Removes irrelevant information that could confuse algorithms
3. **Complexity Reduction:** Simplifies text to focus on meaningful patterns
4. **Performance Enhancement:** Improves the efficiency and accuracy of downstream tasks

Real-world Impact

Consider searching for "running shoes" vs "Running Shoes!" - without preprocessing, these might be treated as completely different queries. Preprocessing ensures they're recognized as equivalent.

🤔 Conceptual Question 1

Before we start coding, think about your daily interactions with text processing systems (search engines, chatbots, translation apps). What challenges do you think these systems face when processing human language? List at least 3 specific challenges and explain why each is problematic.

Double-click this cell to write your answer:

Challenge 1: Spelling mistakes and typing errors. The most common problem is spelling mistake while typing or chating with chatbots and use incorrect words which is hard for system to undrestand.

Challenge 2: People often use slang and abbrevation, so text-preprocessing may misundrstand the words especially on different scinario.

Challenge 3: The same word or sentence can have different meanings depending on the situation. For example, the word "bank" can be financial institution or the side of river. The system needs context to understand the correct meaning.

🛠️ Part 1: Environment Setup

We'll be working with two major NLP libraries:

- **NLTK (Natural Language Toolkit):** Comprehensive NLP library with extensive resources
- **spaCy:** Industrial-strength NLP with pre-trained models

⚠️ **Note:** Installation might take 2-3 minutes to complete.

```
# Step 1: Install Required Libraries
print("🔧 Installing NLP libraries...")

!pip install -q nltk spacy
!python -m spacy download en_core_web_sm

print("✅ Installation complete!")
```

```
🔧 Installing NLP libraries...
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/downl
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 89.9 MB/s e
✔ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Pyth
order to load all the package's dependencies. You can do this by selec
'Restart kernel' or 'Restart runtime' option.
✅ Installation complete!
```

🤔 Conceptual Question 2

Why do you think we need to install a separate language model (en_core_web_sm) for spaCy? What components might this model contain that help with text processing? Think about what information a computer needs to understand English text.

We need to install a separate language model because spaCy itself provide the tools, but the language model contains the information needed to undrstand and process English text. The en_core_web_sm model trained on English and helps spaCy recognise words and their roles in sentences. It contains components for tokenization, part-of-speech tagging, grammar/dependency analysis, lemmatization, and named entityrecognition. these help the computer identify nouns, verbs, people, placesa and relationships between words.

```
# Step 2: Import Libraries and Download NLTK Data
import nltk
import spacy
import string
import re
from collections import Counter

# Download essential NLTK data
print("📦 Downloading NLTK data packages...")
nltk.download('punkt')      # For tokenization
nltk.download('stopwords')  # For stop word removal
nltk.download('wordnet')    # For lemmatization
nltk.download('averaged_perceptron_tagger')  # For POS tagging

print("\n✅ All imports and downloads completed!")
```

```
📦 Downloading NLTK data packages...
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger.zip.

✅ All imports and downloads completed!
```

📂 Part 2: Sample Text Data

We'll work with different types of text to understand how preprocessing affects various text styles:

- Simple text
- Academic text (with citations, URLs)
- Social media text (with emojis, hashtags)
- News text (formal writing)
- Product reviews (informal, ratings)

```
# Step 3: Load Sample Texts
simple_text = "Natural Language Processing is a fascinating field of A
```

```
academic_text = """
Dr. Smith's research on machine-learning algorithms is groundbreaking
She published 3 papers in 2023, focusing on deep neural networks (DNN
The results were amazing - accuracy improved by 15.7%!
"This is revolutionary," said Prof. Johnson.
Visit https://example.com for more info. #NLP #AI @university
"""

social_text = "OMG! Just tried the new coffee shop ☕ SO GOOD!!! High

news_text = """
The stock market experienced significant volatility today, with tech
Apple Inc. (AAPL) dropped 3.2%, while Microsoft Corp. fell 2.8%.
"We're seeing a rotation out of growth stocks," said analyst Jane Doe
"""

review_text = """
This laptop is absolutely fantastic! I've been using it for 6 months
The battery life is incredible - lasts 8-10 hours easily.
Only complaint: the keyboard could be better. Overall rating: 4.5/5 s
"""

# Store all texts
sample_texts = {
    "Simple": simple_text,
    "Academic": academic_text.strip(),
    "Social Media": social_text,
    "News": news_text.strip(),
    "Product Review": review_text.strip()
}

print("📄 Sample texts loaded successfully!")
for name, text in sample_texts.items():
    preview = text[:80] + "..." if len(text) > 80 else text
    print(f"\n📁 {name}: {preview}")
```

```
📄 Sample texts loaded successfully!

📁 Simple: Natural Language Processing is a fascinating field of AI. I

📁 Academic: Dr. Smith's research on machine-learning algorithms is gr
She publi...

📁 Social Media: OMG! Just tried the new coffee shop ☕ SO GOOD!!! Hig

📁 News: The stock market experienced significant volatility today, wi
```

```
Product Review: This laptop is absolutely fantastic! I've been usin
```

🤔 Conceptual Question 3

Looking at the different text types we've loaded, what preprocessing challenges do you anticipate for each type? For each text type below, identify at least 2 specific preprocessing challenges and explain why they might be problematic for NLP analysis.

Double-click this cell to write your answer:

Simple text challenges: One first text "It's" and "AI" are challenges.

1. Punctuation can make tokenization harder.
2. Words can have different meanings depending on context.

Academic text challenges: Second text "Dr.Smith's" and "machine-learning" are challenges.

1. Complex technical terms may be difficult to recognize.
2. Long and complicated sentences can make grammatical analysis harder.

Social media text challenges: Third text has "OMG" and "SO GOOD, emojis and #coffee".

1. Slang and abbreviations like “OMG” may be difficult to understand.
2. Emojis, hashtags, and unusual punctuation like “!!!” require special processing.

News text challenges: Fourth text has complex or formal vocabulary and domain-specfic terminology.

1. Names of people, companies, and places need to be identified correctly.
2. Long sentences and formal vocabulary can make text analysis more complex.

Product review challenges: Fifth has strong opinion statement and contractions.

1. Informal language and spelling mistakes may occur.
2. Sarcasm or mixed opinions can make sentiment difficult to determine.

🔤 Part 3: Tokenization

What is Tokenization?

Tokenization is the process of breaking down text into smaller, meaningful units called **tokens**. These tokens are typically words, but can also be sentences, characters, or subwords.

Why is it Important?

- Most NLP algorithms work with individual tokens, not entire texts
- It's the foundation for all subsequent preprocessing steps
- Different tokenization strategies can significantly impact results

Common Challenges:

- **Contractions:** "don't" → "do" + "n't" or "don't"?
- **Punctuation:** Keep with words or separate?
- **Special characters:** How to handle @, #, URLs?

```
# Step 4: Tokenization with NLTK
from nltk.tokenize import word_tokenize, sent_tokenize
import nltk

# Download punkt_tab if it's explicitly requested by the error
nltk.download('punkt_tab', quiet=True)

# Test on simple text
print("🔍 NLTK Tokenization Results")
print("=" * 40)
print(f"Original: {simple_text}")

# Word tokenization
nltk_tokens = word_tokenize(simple_text)
print(f"\nWord tokens: {nltk_tokens}")
print(f"Number of tokens: {len(nltk_tokens)}")

# Sentence tokenization
sentences = sent_tokenize(simple_text)
print(f"\nSentences: {sentences}")
print(f"Number of sentences: {len(sentences)}")
```

```
🔍 NLTK Tokenization Results
========================================
Original: Natural Language Processing is a fascinating field of AI. It

Word tokens: ['Natural', 'Language', 'Processing', 'is', 'a', 'fascina
Number of tokens: 14

Sentences: ['Natural Language Processing is a fascinating field of AI.
Number of sentences: 2
```

🤔 Conceptual Question 4

Examine the NLTK tokenization results above. How did NLTK handle the contraction "It's"? What happened to the punctuation marks? Do you think this approach is appropriate for all NLP tasks? Explain your reasoning.

Double-click this cell to write your answer:

How "It's" was handled: NLTK split “It’s” into two tokens: “It” and “’s”. This helps the system recognize the contraction as separate parts.

Punctuation treatment: NLTK treated punctuation marks such as “.” and “!” as separate tokens instead of keeping them attached to the words.

Appropriateness for different tasks: This approach is useful for many NLP tasks because it separates words and punctuation for easier analysis. However, it may not be best for every task because punctuation and contractions can sometimes carry important meaning, especially in case of sentiment analysis or social media text.

```
# Step 5: Tokenization with spaCy
nlp = spacy.load('en_core_web_sm')

print("🔍 spaCy Tokenization Results")
print("=" * 40)
print(f"Original: {simple_text}")

# Process with spaCy
doc = nlp(simple_text)

# Extract tokens
spacy_tokens = [token.text for token in doc]
print(f"\nWord tokens: {spacy_tokens}")
print(f"Number of tokens: {len(spacy_tokens)}")

# Show detailed token information
print(f"\n🔬 Detailed Token Analysis:")
print(f"{'Token':<12} {'POS':<8} {'Lemma':<12} {'Is Alpha':<8} {'Is S
print("-" * 50)
for token in doc:
    print(f"{token.text:<12} {token.pos_:<8} {token.lemma_:<12} {toke
```

```
🔍 spaCy Tokenization Results
```

```
🔧 spaCy Tokenization Results
==================================================
Original: Natural Language Processing is a fascinating field of AI. It

Word tokens: ['Natural', 'Language', 'Processing', 'is', 'a', 'fascina
Number of tokens: 14

🔬 Detailed Token Analysis:
Token        POS      Lemma        Is Alpha Is Stop
------------------------------------------------------------
Natural      PROPN    Natural      1        0
Language     PROPN    Language     1        0
Processing   NOUN     processing   1        0
is           AUX      be           1        1
a            DET      a            1        1
fascinating  ADJ      fascinating  1        0
field        NOUN     field        1        0
of           ADP      of           1        1
AI           PROPN    AI           1        0
.            PUNCT    .            0        0
It           PRON     it           1        1
's           AUX      be           0        1
amazing      ADJ      amazing      1        0
!            PUNCT    !            0        0
```

🤔 Conceptual Question 5

Compare the NLTK and spaCy tokenization results. What differences do you notice? Which approach do you think would be better for different NLP tasks? Consider specific examples like sentiment analysis vs. information extraction.

Double-click this cell to write your answer:

Key differences observed: Both NLTK and spaCy split the sentence into 14 tokens and separate “It’s” into “It” and "'s." However, spaCy provides additional information such as part of speech (POS), lemma, stop words, and punctuation identification, while NLTK mainly shows the tokenized words and sentences.

Better for sentiment analysis: NLTK can be better for simple sentiment analysis because it is easy to tokenize and process words to determine whether text is positive or negative.

Better for information extraction: spaCy is better because it provides grammatical information and can identify entities, word relationships, and parts of speech.

Overall assessment: NLTK is useful for simple text processing and learning NLP concepts, while spaCy is more suitable for advanced NLP tasks and information extraction.

```
# Step 6: Test Tokenization on Complex Text
print("🧪 Testing on Social Media Text")
print("=" * 40)
print(f"Original: {social_text}")

# NLTK approach
social_nltk_tokens = word_tokenize(social_text)
print(f"\nNLTK tokens: {social_nltk_tokens}")

# spaCy approach
social_doc = nlp(social_text)
social_spacy_tokens = [token.text for token in social_doc]
print(f"spaCy tokens: {social_spacy_tokens}")

print(f"\n📊 Comparison:")
print(f"NLTK token count: {len(social_nltk_tokens)}")
print(f"spaCy token count: {len(social_spacy_tokens)}")
```

```
🧪 Testing on Social Media Text
========================================
Original: OMG! Just tried the new coffee shop ☕ SO GOOD!!! Highly rec

NLTK tokens: ['OMG', '!', 'Just', 'tried', 'the', 'new', 'coffee', 'sh
spaCy tokens: ['OMG', '!', 'Just', 'tried', 'the', 'new', 'coffee', 'sh

📊 Comparison:
NLTK token count: 22
spaCy token count: 23
```

🤔 Conceptual Question 6

Looking at how the libraries handled social media text (emojis, hashtags), which library seems more robust for handling "messy" real-world text? What specific advantages do you notice? How might this impact a real-world application like social media sentiment analysis?

Double-click this cell to write your answer:

More robust library: spaCy

Specific advantages: spaCy handles messy social media text well. It recognizes emojis, hashtags, punctuation, and unusual symbols as separate tokens more consistently.

Impact on sentiment analysis: Better tokenization helps preserve important information such as emojis and hashtags. For example, 😍 or 👍 can indicate positive sentiment. Keeping these tokens can make social media sentiment analysis more accurate.

🛑 Part 4: Stop Words Removal

What are Stop Words?

Stop words are common words that appear frequently in a language but typically don't carry much meaningful information about the content. Examples include "the", "is", "at", "which", "on", etc.

Why Remove Stop Words?

1. **Reduce noise** in the data
2. **Improve efficiency** by reducing vocabulary size
3. **Focus on content words** that carry semantic meaning

When NOT to Remove Stop Words?

- **Sentiment analysis:** "not good" vs "good" - the "not" is crucial!
- **Question answering:** "What is the capital?" - "what" and "is" provide context

```
# Step 7: Explore Stop Words Lists
from nltk.corpus import stopwords

# Get NLTK English stop words
nltk_stopwords = set(stopwords.words('english'))
print(f"📊 NLTK has {len(nltk_stopwords)} English stop words")
print(f"First 20: {sorted(list(nltk_stopwords))[:20]}")

# Get spaCy stop words
spacy_stopwords = nlp.Defaults.stop_words
print(f"\n📊 spaCy has {len(spacy_stopwords)} English stop words")
print(f"First 20: {sorted(list(spacy_stopwords))[:20]}")

# Compare the lists
common_stopwords = nltk_stopwords.intersection(spacy_stopwords)
nltk_only = nltk_stopwords - spacy_stopwords
spacy_only = spacy_stopwords - nltk_stopwords

print(f"\n🔍 Comparison:")
print(f"Common stop words: {len(common_stopwords)}")
print(f"Only in NLTK: {len(nltk_only)} - Examples: {sorted(list(nltk_
print(f"Only in spaCy: {len(spacy_only)} - Examples: {sorted(list(spa
```

```
📊 NLTK has 198 English stop words
First 20: ['a', 'about', 'above', 'after', 'again', 'against', 'ain',

📊 spaCy has 326 English stop words
First 20: ["'d", "'ll", "'m", "'re", "'s", "'ve", 'a', 'about', 'above

🔍 Comparison:
Common stop words: 123
Only in NLTK: 75 - Examples: ['ain', 'aren', "aren't", 'couldn', "coul
Only in spaCy: 203 - Examples: ["'d", "'ll", "'m", "'re", "'s"]
```

🤔 Conceptual Question 7

Why do you think NLTK and spaCy have different stop word lists? Look at the examples of words that are only in one list - do you agree with these choices? Can you think of scenarios where these differences might significantly impact your NLP results?

Double-click this cell to write your answer:

Reasons for differences: NLTK and spaCy use different rules and were designed differently. spaCy has a larger stop-word list (326 words), while NLTK has 198.

Agreement with choices: Mostly yes. However, some words should be kept depending on the NLP task because they may carry important meaning.

Scenarios where differences matter: In sentiment analysis, removing words like “not,” “no,” or “never” can change the meaning of a sentence. For example, “I like this” and “I do not like this” have opposite meanings. Therefore, choosing the correct stop-word list can significantly affect NLP results.

```
# Step 8: Remove Stop Words with NLTK
# Test on simple text
original_tokens = nltk_tokens  # From earlier tokenization
filtered_tokens = [word for word in original_tokens if word.lower() n

print("🧪 NLTK Stop Word Removal")
print("=" * 40)
print(f"Original: {simple_text}")
print(f"\nOriginal tokens ({len(original_tokens)}): {original_tokens}'
print(f"After removing stop words ({len(filtered_tokens)}): {filtered_

# Show which words were removed
removed_words = [word for word in original_tokens if word.lower() in r
print(f"\nRemoved words: {removed_words}")

# Calculate reduction percentage
reduction = (len(original_tokens) - len(filtered_tokens)) / len(origi
print(f"Vocabulary reduction: {reduction:.1f}%")
```

```
🧪 NLTK Stop Word Removal
========================================
Original: Natural Language Processing is a fascinating field of AI. It

Original tokens (14): ['Natural', 'Language', 'Processing', 'is', 'a',
After removing stop words (10): ['Natural', 'Language', 'Processing',

Removed words: ['is', 'a', 'of', 'It']
Vocabulary reduction: 28.6%
```

```
# Step 9: Remove Stop Words with spaCy
doc = nlp(simple_text)
spacy_filtered = [token.text for token in doc if not token.is_stop and

print("🧪 spaCy Stop Word Removal")
print("=" * 40)
print(f"Original: {simple_text}")
print(f"\nOriginal tokens ({len(spacy_tokens)}): {spacy_tokens}")
print(f"After removing stop words & punctuation ({len(spacy_filtered)}

# Show which words were removed
spacy_removed = [token.text for token in doc if token.is_stop or token
print(f"\nRemoved words: {spacy_removed}")

# Calculate reduction percentage
spacy_reduction = (len(spacy_tokens) - len(spacy_filtered)) / len(spac
print(f"Vocabulary reduction: {spacy_reduction:.1f}%")
```

```
🧪 spaCy Stop Word Removal
========================================
Original: Natural Language Processing is a fascinating field of AI. It

Original tokens (14): ['Natural', 'Language', 'Processing', 'is', 'a',
After removing stop words & punctuation (7): ['Natural', 'Language', 'P

Removed words: ['is', 'a', 'of', '.', 'It', "'s", '!']
Vocabulary reduction: 50.0%
```

🤔 Conceptual Question 8

Compare the NLTK and spaCy stop word removal results. Which approach removed more words? Do you think removing punctuation (as spaCy did) is always a good idea? Give a specific example where keeping punctuation might be important for NLP analysis.

Double-click this cell to write your answer:

Which removed more: spaCy removed more. It reduced the vocabulary by 50%, while NLTK reduced it by 28.6%.

Punctuation removal assessment: Removing punctuation can make text cleaner, but it is not always a good idea because punctuation can provide information about emotion, tone, or meaning.

Example where punctuation matters: In sentiment analysis, “This is amazing!” shows stronger positive emotion than “This is amazing.” The exclamation mark helps show excitement.

🌱 Part 5: Lemmatization and Stemming

What is Lemmatization?

Lemmatization reduces words to their base or dictionary form (called a **lemma**). It considers context and part of speech to ensure the result is a valid word.

What is Stemming?

Stemming reduces words to their root form by removing suffixes. It's faster but less accurate than lemmatization.

Key Differences:

Aspect	Stemming	Lemmatization
Speed	Fast	Slower
Accuracy	Lower	Higher
Output	May be non-words	Always valid words
Context	Ignores context	Considers context

Examples:

- **"running"** → Stem: "run", Lemma: "run"
- **"better"** → Stem: "better", Lemma: "good"
- **"was"** → Stem: "wa", Lemma: "be"

```
# Step 10: Stemming with NLTK
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

# Test words that demonstrate stemming challenges
test_words = ['running', 'runs', 'ran', 'better', 'good', 'best', 'fly

print("🌿 Stemming Demonstration")
print("=" * 30)
print(f"{'Original':<12} {'Stemmed':<12}")
print("-" * 25)

for word in test_words:
    stemmed = stemmer.stem(word)
    print(f"{word:<12} {stemmed:<12}")

# Apply to our sample text
sample_tokens = [token for token in nltk_tokens if token.isalpha()]
stemmed_tokens = [stemmer.stem(token.lower()) for token in sample_tok

print(f"\n🧪 Applied to sample text:")
print(f"Original: {sample_tokens}")
print(f"Stemmed: {stemmed_tokens}")
```

```
🌿 Stemming Demonstration
==============================
Original     Stemmed
-------------------------
running      run
runs         run
ran          ran
better       better
good         good
best         best
flying       fli
flies        fli
was          wa
were         were
cats         cat
dogs         dog

🧪 Applied to sample text:
Original: ['Natural', 'Language', 'Processing', 'is', 'a', 'fascinatin
Stemmed: ['natur', 'languag', 'process', 'is', 'a', 'fascin', 'field',
```

🤔 Conceptual Question 9

Look at the stemming results above. Can you identify any cases where stemming produced questionable results? For example, how were "better" and "good" handled? Do you think this is problematic for NLP applications? Explain your reasoning.

Double-click this cell to write your answer:

Questionable results identified: Some stemming results are there, such as “flying” → “fli” and “flies” → “fli.” “Was” also becomes “wa,” which is not a real word.

Assessment of "better" and "good": They are not handled well because “better” remains “better” instead of becoming “good.” Stemming does not understand the actual meaning of words.

Impact on NLP applications: This can be problematic because incorrect or unnatural stems may cause the system to misunderstand words. Lemmatization is often better when preserving the correct meaning is important.

```
# Step 11: Lemmatization with spaCy
print("🌱 spaCy Lemmatization Demonstration")
print("=" * 40)

# Test on a complex sentence
complex_sentence = "The researchers were studying the effects of runn
doc = nlp(complex_sentence)

print(f"Original: {complex_sentence}")
print(f"\n{'Token':<15} {'Lemma':<15} {'POS':<10} {'Explanation':<20}"
print("-" * 65)

for token in doc:
    if token.is_alpha:
        explanation = "No change" if token.text.lower() == token.lemma
        print(f"{token.text:<15} {token.lemma_:<15} {token.pos_:<10} {

# Extract lemmas
lemmas = [token.lemma_.lower() for token in doc if token.is_alpha and
print(f"\n🔤 Lemmatized tokens (no stop words): {lemmas}")
```

```
🌱 spaCy Lemmatization Demonstration
========================================
Original: The researchers were studying the effects of running and swi

Token           Lemma           POS        Explanation
-----------------------------------------------------------------
The             the             DET        No change
researchers     researcher      NOUN       Lemmatized
were            be              AUX        Lemmatized
studying        study           VERB       Lemmatized
the             the             DET        No change
effects         effect          NOUN       Lemmatized
of              of              ADP        No change
running         run             VERB       Lemmatized
and             and             CCONJ      No change
swimming        swim            VERB       Lemmatized
on              on              ADP        No change
better          well            ADJ        Lemmatized
performance     performance     NOUN       No change

🔤 Lemmatized tokens (no stop words): ['researcher', 'study', 'effect'
```

```
# Step 12: Compare Stemming vs Lemmatization
comparison_words = ['better', 'running', 'studies', 'was', 'children',

print("⚖️ Stemming vs Lemmatization Comparison")
print("=" * 50)
print(f"{'Original':<12} {'Stemmed':<12} {'Lemmatized':<12}")
print("-" * 40)

for word in comparison_words:
    # Stemming
    stemmed = stemmer.stem(word)

    # Lemmatization with spaCy
    doc = nlp(word)
    lemmatized = doc[0].lemma_

    print(f"{word:<12} {stemmed:<12} {lemmatized:<12}")
```

```
⚖️ Stemming vs Lemmatization Comparison
==================================================
Original     Stemmed      Lemmatized
----------------------------------------
better       better       well
running      run          run
studies      studi        study
was          wa           be
children     children     child
feet         feet         foot
```

🤔 Conceptual Question 10

Compare the stemming and lemmatization results. Which approach do you think is more suitable for:

1. **A search engine** (where speed is crucial and you need to match variations of words)?
2. **A sentiment analysis system** (where accuracy and meaning preservation are important)?
3. **A real-time chatbot** (where both speed and accuracy matter)?

Explain your reasoning for each choice.

Double-click this cell to write your answer:

1. Search engine: Stemming is more suitable because it is faster and can quickly match different forms of a word. For example, “running” and “runs” can both become “run.”

2. Sentiment analysis: Lemmatization is better because it preserves the actual meaning of words. For example, it correctly changes “better” to “well,” while stemming leaves it as “better.”

3. Real-time chatbot: Lemmatization is more suitable because understanding the correct meaning is very important for giving accurate responses. Modern NLP libraries like spaCy can perform lemmatization efficiently enough for real-time applications.

🧹 Part 6: Text Cleaning and Normalization

What is Text Cleaning?

Text cleaning involves removing or standardizing elements that might interfere with analysis:

- **Case normalization** (converting to lowercase)
- **Punctuation removal**
- **Number handling** (remove, replace, or normalize)
- **Special character handling** (URLs, emails, mentions)
- **Whitespace normalization**

Why is it Important?

- Ensures consistency across your dataset
- Reduces vocabulary size
- Improves model performance
- Handles edge cases in real-world data

```
# Step 13: Basic Text Cleaning
def basic_clean_text(text):
    """Apply basic text cleaning operations"""
    # Convert to lowercase
    text = text.lower()

    # Remove extra whitespace
    text = re.sub(r'\s+', ' ', text).strip()

    # Remove punctuation
    text = text.translate(str.maketrans('', '', string.punctuation))

    # Remove numbers
    text = re.sub(r'\d+', '', text)

    # Remove extra spaces again
    text = re.sub(r'\s+', ' ', text).strip()

    return text

# Test basic cleaning
test_text = "   Hello WORLD!!! This has 123 numbers and   extra spaces
cleaned = basic_clean_text(test_text)

print("🧹 Basic Text Cleaning")
print("=" * 30)
print(f"Original: '{test_text}'")
print(f"Cleaned: '{cleaned}'")
print(f"Length reduction: {(len(test_text) - len(cleaned))/len(test_te
```

```
🧹 Basic Text Cleaning
==============================
Original: '   Hello WORLD!!! This has 123 numbers and   extra spaces.
Cleaned: 'hello world this has numbers and extra spaces'
Length reduction: 26.2%
```

```
# Step 14: Advanced Cleaning for Social Media
def advanced_clean_text(text):
    """Apply advanced cleaning for social media and web text"""
    # Remove URLs
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTI

    # Remove email addresses
    text = re.sub(r'\S+@\S+', '', text)
```

```
    # Remove mentions (@username)
    text = re.sub(r'@\w+', '', text)

    # Convert hashtags (keep the word, remove #)
    text = re.sub(r'#(\w+)', r'\1', text)

    # Remove emojis (basic approach)
    emoji_pattern = re.compile("["
                               u"\U0001F600-\U0001F64F"  # emoticons
                               u"\U0001F300-\U0001F5FF"  # symbols & p
                               u"\U0001F680-\U0001F6FF"  # transport &
                               u"\U0001F1E0-\U0001F1FF"  # flags
                               "]+", flags=re.UNICODE)
    text = emoji_pattern.sub(r'', text)

    # Convert to lowercase and normalize whitespace
    text = text.lower()
    text = re.sub(r'\s+', ' ', text).strip()

    return text

# Test on social media text
print("🚀 Advanced Cleaning on Social Media Text")
print("=" * 45)
print(f"Original: {social_text}")

cleaned_social = advanced_clean_text(social_text)
print(f"Cleaned: {cleaned_social}")
print(f"Length reduction: {(len(social_text) - len(cleaned_social))/le
```

```
🚀 Advanced Cleaning on Social Media Text
=============================================
Original: OMG! Just tried the new coffee shop ☕ SO GOOD!!! Highly rec
Cleaned: omg! just tried the new coffee shop ☕ so good!!! highly reco
Length reduction: 7.2%
```

🤔 Conceptual Question 11

Look at the advanced cleaning results for the social media text. What information was lost during cleaning? Can you think of scenarios where removing emojis and hashtags might actually hurt your NLP application? What about scenarios where keeping them would be beneficial?

Double-click this cell to write your answer:

Information lost: The cleaning removed emojis such as 👍 and 😍 and removed the # symbols from hashtags. This can remove information about emotion, emphasis, and topics.

Scenarios where removal hurts: Removing them can hurt sentiment analysis because emojis often show whether someone feels positive or negative. Hashtags can also identify important topics or trends.

Scenarios where keeping helps: Keeping emojis and hashtags is useful for social media analysis, sentiment detection, and trend analysis because they provide extra information about the other feelings and the topic being discussed.

🔧 Part 7: Building a Complete Preprocessing Pipeline

Now let's combine everything into a comprehensive preprocessing pipeline that you can customize based on your needs.

Pipeline Components:

1. **Text cleaning** (basic or advanced)
2. **Tokenization** (NLTK or spaCy)
3. **Stop word removal** (optional)
4. **Lemmatization/Stemming** (optional)
5. **Additional filtering** (length, etc.)

```
# Step 15: Complete Preprocessing Pipeline
def preprocess_text(text,
                    clean_level='basic',     # 'basic' or 'advanced'
```

```
                    remove_stopwords=True,
                    use_lemmatization=True,
                    use_stemming=False,
                    min_length=2):
    """
    Complete text preprocessing pipeline
    """
    # Step 1: Clean text
    if clean_level == 'basic':
        cleaned_text = basic_clean_text(text)
    else:
        cleaned_text = advanced_clean_text(text)

    # Step 2: Tokenize
    if use_lemmatization:
        # Use spaCy for lemmatization
        doc = nlp(cleaned_text)
        tokens = [token.lemma_.lower() for token in doc if token.is_a
    else:
        # Use NLTK for basic tokenization
        tokens = word_tokenize(cleaned_text)
        tokens = [token for token in tokens if token.isalpha()]

    # Step 3: Remove stop words
    if remove_stopwords:
        if use_lemmatization:
            tokens = [token for token in tokens if token not in spacy_
        else:
            tokens = [token.lower() for token in tokens if token.lowe

    # Step 4: Apply stemming if requested
    if use_stemming and not use_lemmatization:
        tokens = [stemmer.stem(token.lower()) for token in tokens]

    # Step 5: Filter by length
    tokens = [token for token in tokens if len(token) >= min_length]

    return tokens

print("🔧 Preprocessing Pipeline Created!")
print("✅ Ready to test different configurations.")
```

```
🔧 Preprocessing Pipeline Created!
✅ Ready to test different configurations.
```

```
# Step 16: Test Different Pipeline Configurations
test_text = sample_texts["Product Review"]
print(f"🎯 Testing on: {test_text[:100]}...")
print("=" * 60)

# Configuration 1: Minimal processing
minimal = preprocess_text(test_text,
                          clean_level='basic',
                          remove_stopwords=False,
                          use_lemmatization=False,
                          use_stemming=False)
print(f"\n1. Minimal processing ({len(minimal)} tokens):")
print(f"   {minimal[:10]}...")

# Configuration 2: Standard processing
standard = preprocess_text(test_text,
                           clean_level='basic',
                           remove_stopwords=True,
                           use_lemmatization=True)
print(f"\n2. Standard processing ({len(standard)} tokens):")
print(f"   {standard[:10]}...")

# Configuration 3: Aggressive processing
aggressive = preprocess_text(test_text,
                             clean_level='advanced',
                             remove_stopwords=True,
                             use_lemmatization=False,
                             use_stemming=True,
                             min_length=3)
print(f"\n3. Aggressive processing ({len(aggressive)} tokens):")
print(f"   {aggressive[:10]}...")

# Show reduction percentages
original_count = len(word_tokenize(test_text))
print(f"\n📊 Token Reduction Summary:")
print(f"   Original: {original_count} tokens")
print(f"   Minimal: {len(minimal)} ({(original_count-len(minimal))/or
print(f"   Standard: {len(standard)} ({(original_count-len(standard)),
print(f"   Aggressive: {len(aggressive)} ({(original_count-len(aggres
```

```
🎯 Testing on: This laptop is absolutely fantastic! I've been using it
The ...
============================================================

1. Minimal processing (34 tokens):
```

```
   ['this', 'laptop', 'is', 'absolutely', 'fantastic', 'ive', 'been',

2. Standard processing (18 tokens):
   ['laptop', 'absolutely', 'fantastic', 've', 'use', 'month', 'super'

3. Aggressive processing (21 tokens):
   ['laptop', 'absolut', 'fantast', 'use', 'month', 'still', 'super',

📊 Token Reduction Summary:
   Original: 47 tokens
   Minimal: 34 (27.7% reduction)
   Standard: 18 (61.7% reduction)
   Aggressive: 21 (55.3% reduction)
```

🤔 Conceptual Question 12

Compare the three pipeline configurations (Minimal, Standard, Aggressive). For each configuration, analyze:

1. **What information was preserved?**
2. **What information was lost?**
3. **What type of NLP task would this configuration be best suited for?**

Double-click this cell to write your answer:

Minimal Processing:

- Preserved: Most original words and sentence information are preserved.
- Lost: Very little information is lost; mainly basic formatting or unnecessary characters.
- Best for: Sentiment analysis or chatbots, where context and meaning are important.

Standard Processing:

- Preserved: Important words and their base meanings are preserved using lemmatization.
- Lost: Stop words, punctuation, and some unnecessary words are removed.
- Best for: Text classification and topic analysis, because it balances cleaning with meaning preservation.

Aggressive Processing:

- Preserved: Mainly the important root forms of words are preserved.
- Lost: More grammatical and contextual information is lost because stemming can produce incomplete words like “fantast” and “batteri.”
- Best for: Search engines and keyword matching, where reducing words to similar forms is more important than preserving perfect grammar.

```
# Step 17: Comprehensive Analysis Across Text Types
print("🔬 Comprehensive Preprocessing Analysis")
```

```
print("=" * 50)

# Test standard preprocessing on all text types
results = {}
for name, text in sample_texts.items():
    original_tokens = len(word_tokenize(text))
    processed_tokens = preprocess_text(text,
                                       clean_level='basic',
                                       remove_stopwords=True,
                                       use_lemmatization=True)

    reduction = (original_tokens - len(processed_tokens)) / original_
    results[name] = {
        'original': original_tokens,
        'processed': len(processed_tokens),
        'reduction': reduction,
        'sample': processed_tokens[:8]
    }

    print(f"\n📄 {name}:")
    print(f"   Original: {original_tokens} tokens")
    print(f"   Processed: {len(processed_tokens)} tokens ({reduction:
    print(f"   Sample: {processed_tokens[:8]}")

# Summary table
print(f"\n\n📋 Summary Table")
print(f"{'Text Type':<15} {'Original':<10} {'Processed':<10} {'Reduct
print("-" * 50)
for name, data in results.items():
    print(f"{name:<15} {data['original']:<10} {data['processed']:<10}
```

```
🔬 Comprehensive Preprocessing Analysis
==================================================

📄 Simple:
   Original: 14 tokens
   Processed: 7 tokens (50.0% reduction)
   Sample: ['natural', 'language', 'processing', 'fascinating', 'field

📄 Academic:
   Original: 61 tokens
   Processed: 26 tokens (57.4% reduction)
   Sample: ['dr', 'smith', 'research', 'machinelearning', 'algorithm',

📄 Social Media:
   Original: 22 tokens
```

```
   Processed: 10 tokens (54.5% reduction)
   Sample: ['omg', 'try', 'new', 'coffee', 'shop', 'good', 'highly', '

📄 News:
   Original: 51 tokens
   Processed: 25 tokens (51.0% reduction)
   Sample: ['stock', 'market', 'experience', 'significant', 'volatilit

📄 Product Review:
   Original: 47 tokens
   Processed: 18 tokens (61.7% reduction)
   Sample: ['laptop', 'absolutely', 'fantastic', 've', 'use', 'month',


📋 Summary Table
Text Type       Original   Processed  Reduction
------------------------------------------------
Simple          14         7          50.0      %
Academic        61         26         57.4      %
Social Media    22         10         54.5      %
News            51         25         51.0      %
Product Review  47         18         61.7      %
```

🤔 Final Conceptual Question 13

Looking at the comprehensive analysis results across all text types:

1. **Which text type was most affected by preprocessing?** Why do you think this happened?
2. **Which text type was least affected?** What does this tell you about the nature of that text?
3. **If you were building an NLP system to analyze customer reviews for a business, which preprocessing approach would you choose and why?**
4. **What are the main trade-offs you need to consider when choosing preprocessing techniques for any NLP project?**

Double-click this cell to write your answer:

1. Most affected text type: Product Review was most affected, with a 61.7% reduction. This happened because reviews contain many stop words, punctuation, and different word forms that preprocessing removes or simplifies.

2. Least affected text type: Simple text was least affected, with a 50% reduction. This suggests that the text was already short, clear, and contained less unnecessary information.

3. For customer review analysis: I would choose standard preprocessing with lemmatization because it removes unnecessary words while preserving the meaning of important words. This is useful for accurately identifying customer opinions and sentiment.

4. Main trade-offs to consider: The main trade-off is between reducing unnecessary data and preserving useful meaning. Aggressive preprocessing can make processing faster and reduce vocabulary, but it may remove important context, emotion, punctuation, or word meaning. The best preprocessing method depends on the specific NLP task.

🎯 Lab Summary and Reflection

Congratulations! You've completed a comprehensive exploration of NLP preprocessing techniques.

🔑 Key Concepts You've Mastered:

1. **Text Preprocessing Fundamentals** - Understanding why preprocessing is crucial
2. **Tokenization Techniques** - NLTK vs spaCy approaches and their trade-offs
3. **Stop Word Management** - When to remove them and when to keep them
4. **Morphological Processing** - Stemming vs lemmatization for different use cases
5. **Text Cleaning Strategies** - Basic vs advanced cleaning for different text types
6. **Pipeline Design** - Building modular, configurable preprocessing systems

🎓 Real-World Applications:

These techniques form the foundation for search engines, chatbots, sentiment analysis, document classification, machine translation, and information extraction systems.

💡 Key Insights to Remember:

- **No Universal Solution**: Different NLP tasks require different preprocessing approaches
- **Trade-offs Are Everywhere**: Balance information preservation with noise reduction
- **Context Matters**: The same technique can help or hurt depending on your use case
- **Experimentation Is Key**: Always test and measure impact on your specific task

---

Excellent work completing Lab 02! 🎉

For your reflection journal, focus on the insights you gained about when and why to use different techniques, the challenges you encountered, and connections you made to real-world applications.